Automated Verification of ER Diagrams from Natural Language Requirements

Nikitha Reddy Baddam,

School of Computing, University of Nebraska-Lincoln, 1400
R St, Lincoln, NE, USA

, nbaddam2@huskers.unl.edu

ABSTRACT

In this mini project, we explore how to automatically check whether a natural language description matches an Entity–Relationship (ER) diagram written in PlantUML using Chen's notation. We start with a software requirements dataset on Kaggle and sample requirement sentences as our raw input. Then we use a large language model (GPT-5.1) to generate three kinds of labeled examples: PERFECT, SYNONYM, and LOGIC_ERROR, so we get a small but clear dataset. On top of this dataset, we evaluate four methods for MATCH/MISMATCH classification: two rule-based baselines, one embedding-based semantic matching baseline, and one LLM-based semantic and structural reasoning method, which serves as our main approach. Experimental results show that the LLM-based method is more robust to synonym variation and better at detecting logical inconsistencies compared to the baselines, highlighting the potential of LLMs for requirement–diagram consistency checking.

1. INTRODUCTION

In real projects, business analysts or domain experts often write requirement documents, while database designers draw ER diagrams based on these texts. If the requirement and the ER diagram are inconsistent, the implemented system may not satisfy the real needs and fixing this later can be expensive [1]. Now, most consistency checking between requirements and ER diagrams is still done manually: someone reads the text, looks at the diagram, and decides whether they match or not. This is time-consuming and easy to make mistakes. With the rapid development of large language models (LLMs), there is a natural question: can we use LLMs to help check whether a requirement and its ER diagram are consistent [2][3]

In this mini project, we focus on a simple but clear task. For each example, we have (1) one natural language requirement sentence, and (2) one ER diagram written in PlantUML using Chen's notation [4]. Our goal is to build a function that takes this pair as input and outputs a label: MATCH or MISMATCH. Intuitively, MATCH means that the entities and relationships in the ER diagram are reasonable for the requirement. MISMATCH means there is some obvious inconsistency. To study this problem, we build three small datasets (PERFECT, SYNONYM, LOGIC_ERROR) where each requirement–diagram pair has a label as the ground truth.

There are some challenges. First, natural language requirements are flexible: the same meaning can be expressed with different words, and people use many synonyms and variations. Second, simple rule-based or keyword matching methods are very sensitive to naming changes and may fail when the words are different, but the meaning is the same. Finally, logical errors may not change many words, so they are hard to detect by surface similarity alone. These challenges motivate us to design different types of methods and see how they behave on our datasets.

More specifically, our objectives are:
(1) to build an automatic data generation pipeline that starts from a Kaggle requirements dataset and, with the help of GPT-5.1, creates labeled requirement–ER diagram pairs of different types (PERFECT, SYNONYM, LOGIC_ERROR);
(2) to implement four different methods that take a requirement and a PlantUML ER diagram as input and classify them as MATCH or MISMATCH, where three methods are used as baselines and our main method is an LLMbased reasoning approach [5][6];

(3) to run these methods on the same dataset and observe how they behave on perfect matches, synonym changes, and logical errors.

2. DATA GENERATION

2.1 Dataset and Preprocessing

Our data generation process starts from a real software requirements dataset downloaded from Kaggle [7]. The original CSV file is stored locally in the project under kaggle data/system_requirements_5000.csv. This file contains several thousand short requirement sentences.

Before generating any ER diagrams, we perform some simple preprocessing steps. We drop duplicate requirement sentences so that the same text will not appear multiple times in our generated datasets. After this preprocessing, we obtain a cleaned list of unique requirement sentences, which serves as the input pool for the later generation steps. From this pool, we randomly select 20 requirements as the base requirements for all three datasets.

2.2 Generation Pipeline

Based on the cleaned requirement list, we built an automatic generation pipeline. The overall idea is illustrated in Figure 1.

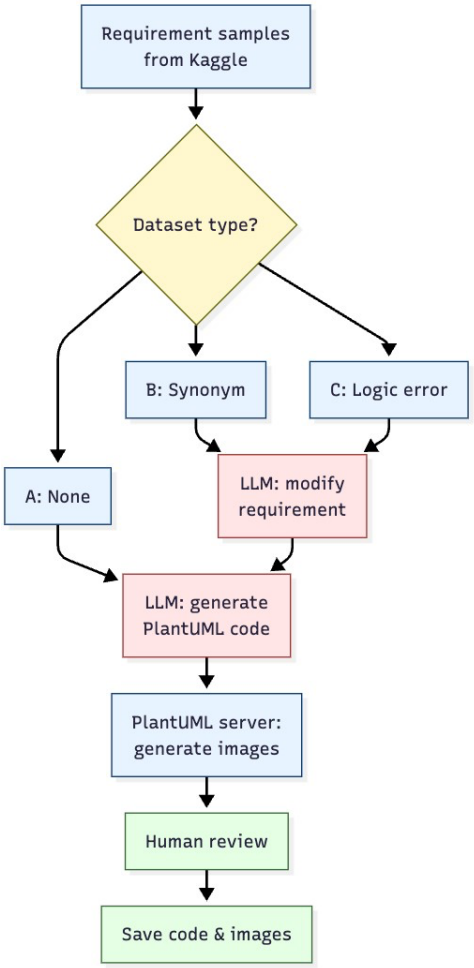


Figure 1. Dataset generation pipeline.

The pipeline works as follows for each selected base requirement sentence:

(1) We start from the original requirement text.

(2) Using GPT-5.1 with different prompts, we create three textual versions:

- the original requirement (used for Dataset A: PERFECT),

- a synonym-modified requirement where 2–4 key terms are replaced by meaningful synonyms (used for Dataset B: SYNONYM), the prompt is shown in Figure 2.
- a requirement with 2–3 clear logical errors intentionally injected (used for Dataset C: LOGIC_ERROR), the prompt is shown in Figure 3.

```
    prompt = f"""Given this requirement text:
{requirement_text}

Task: Replace 2-4 key terms (entity names, attribute names, or domain/role terms) with MEANINGFUL SYNONYMS.

IMPORTANT: Make SUBSTANTIAL and VARIED replacements, not minor ones. Examples of good replacements:
- Staff→Manager, Staff→Personnel, Staff→Employee, Staff→Operator
- Doctor→Physician, Doctor→Practitioner, Doctor→Clinician, Doctor→Specialist
- Book→Volume, Book→Publication, Book→Work, Book→Item
- Patient→Client, Patient→Individual, Patient→Person, Patient→Recipient
- Hospital→Medical Center, Hospital→Clinic, Hospital→Facility, Hospital→Institution
- Add→Insert, Add→Register, Add→Create, Add→Submit
- Email→E-mail address, Email→Electronic mail, Email→Contact info
- Schedule→Arrange, Schedule→Plan, Schedule→Book, Schedule→Assign

Rules:
1. REPLACE 2-4 terms with CLEAR, MEANINGFUL SYNONYMS (not trivial variations)
2. The core meaning must stay the SAME (genuine synonyms/close alternatives)
3. Choose terms that would be found in real domain documents
4. Make replacements that are NOTICEABLY DIFFERENT, not just minor variations
5. Keep the requirement structure and grammar intact
6. Output ONLY the modified requirement text (no explanations, no commentary)
7. Maintain all other details exactly as they are
"""
```

Figure 2. Prompt to generate a synonym-modified requirement.

```
    prompt = f"""Given this requirement text:
{requirement_text}

Task: Introduce 2-3 CLEAR LOGIC ERRORS that make the requirement obviously WRONG.

Error examples (choose different types):
1. Reverse WHO does WHAT: "teachers instruct students" → "students instruct teachers"
2. Change the ACTION to something illogical: "author writes book" → "book writes author"
3. Flip ONE/MANY incorrectly: "one course has many students" → "one student has many courses" (when illogical)
4. Reverse the direction: "employee reports to manager" → "manager reports to employee"
5. Change verb to opposite meaning: "doctor treats patient" → "patient treats doctor"

CRITICAL: The errors must be OBVIOUSLY WRONG when you read them.
Someone should immediately notice "that doesn't make sense!"

Requirements:
- Make 2-3 changes minimum
- Each change should make the logic clearly INCORRECT
- Keep similar sentence length
- The modified text should sound wrong/illogical
- Output ONLY the modified requirement text (no explanations)

Example:
Original: "The system shall allow teachers to assign homework to students."
Wrong: "The system shall allow students to assign homework to teachers."
(This is CLEARLY backwards!)
"""
```

Figure 3. Prompt to generate a logical error requirement.

(3) We use GPT-5.1 again with a specialized prompt to generate a PlantUML ER diagram in Chen's notation. The prompt asks the model to extract entities, attributes, and relationships from the text and to produce valid PlantUML code.

(4) We then send the PlantUML code to an online rendering service [8][9] to render the image. This image, together with the requirement text and PlantUML code, will be used later in our experiments.

(5) Finally, we write all relevant information into CSV files, including the requirement text, the generated PlantUML code, the file path of the PNG image, and a label that indicates whether the pair is a MATCH or a MISMATCH.

By repeating this pipeline for all selected requirements, we obtain three datasets: Dataset A (PERFECT), Dataset B (SYNONYM), and Dataset C (LOGIC_ERROR), all constructed from the same base requirement pool but with different transformations. Table 1 shows the statistics about this dataset

Table 1. Datasets Structure.

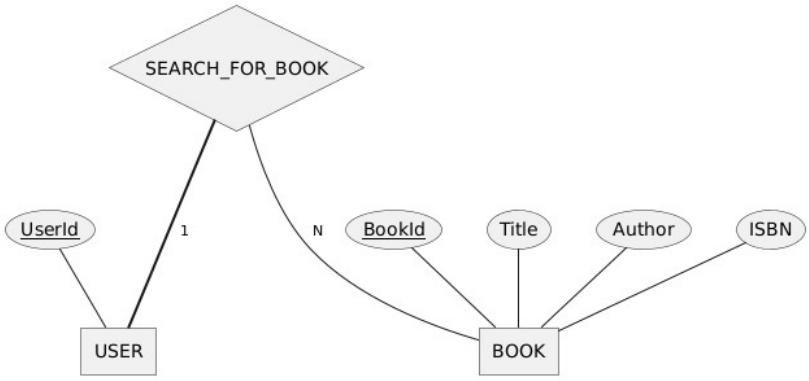
Dataset	Mutation	Label	Number
A – Perfect	None	MATCH	20
B – Synonym	2-4 terms	MATCH	20
C – Logic Error	2-3 errors	MISMATCH	20
Total			60

2.3 Example Samples.

To make the datasets more concrete, we show one simple requirement and its variants. Here is one original Requirement: "Users shall be able to search for books by title, author, or ISBN."

2.3.1 Dataset A – Perfect Match (MATCH)

In Dataset A, we do not modify the requirement text. The requirement is still:
"Users shall be able to search for books by title, author, or ISBN."
This example is labelled as MATCH because the ER diagram reasonably follows the original requirement, which is shown in Figure 4.



Error Type: none
Label: MATCH

Original: Users shall be able to search for books by title, author, or ISBN.

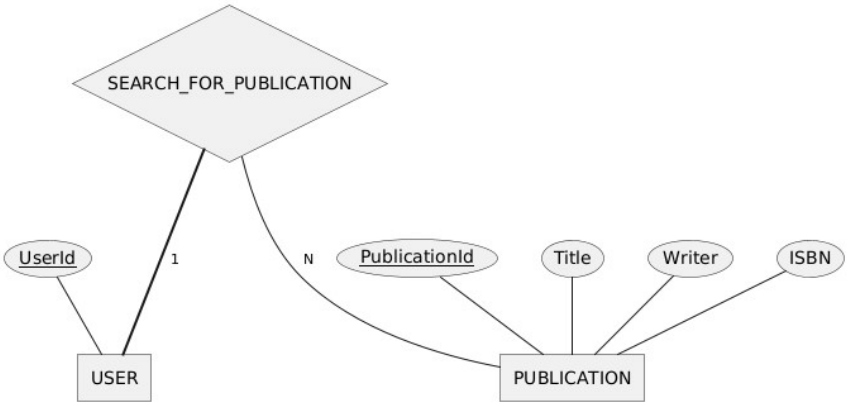
PlantUML Code (sample):

```
@startchen
entity USER {
  UserId <<key>>
}
entity BOOK {
  BookId <<key>>
  Title
  Author
  ISBN
}
relationship SEARCH_FOR_BOOK {
}
SEARCH_FOR_BOOK =1= USER
SEARCH_FOR_BOOK -N- BOOK
@endchen
```

Figure 4. Example of Dataset A, Match

2.3.2 Dataset B – Synonym (MATCH)

In Dataset B, we apply a synonym change. For example, the requirement is changed to:
"Users shall be able to search for publications by title, writer, or ISBN."
Here, "books" is replaced by "publications" and "author" is replaced by "writer", but the meaning is still almost the same. The ER diagram is generated from this synonym-modified requirement, so the structure of the diagram is still consistent with the original intent. Even though the wording is different, we still treat this as a MATCH example, which is shown in Figure 5.



Error Type: synonym
Label: MATCH

Original: Users shall be able to search for books by title, author, or ISBN.
Changed: Users shall be able to search for publications by title, writer, or ISBN.

PlantUML Code (sample):

```
@startchen
entity USER {
  UserId <<key>>
}
entity PUBLICATION {
  PublicationId <<key>>
  Title
  Writer
  ISBN
}
relationship SEARCH_FOR_PUBLICATION {
}
SEARCH_FOR_PUBLICATION =1= USER
SEARCH_FOR_PUBLICATION -N- PUBLICATION
@endchen
```

Figure 5. Example of Dataset B, Match

2.3.3 Dataset C – Logic Error (MISMATCH)

In Dataset C, we introduce logical errors into the requirement. For example, the requirement is changed to: "Users shall be able to hide all books by title, replace authors with ISBNs, and delete search results permanently."
Now the actions described are very different from the original one, and this pair should be labeled as MISMATCH, which is shown in Figure 6.

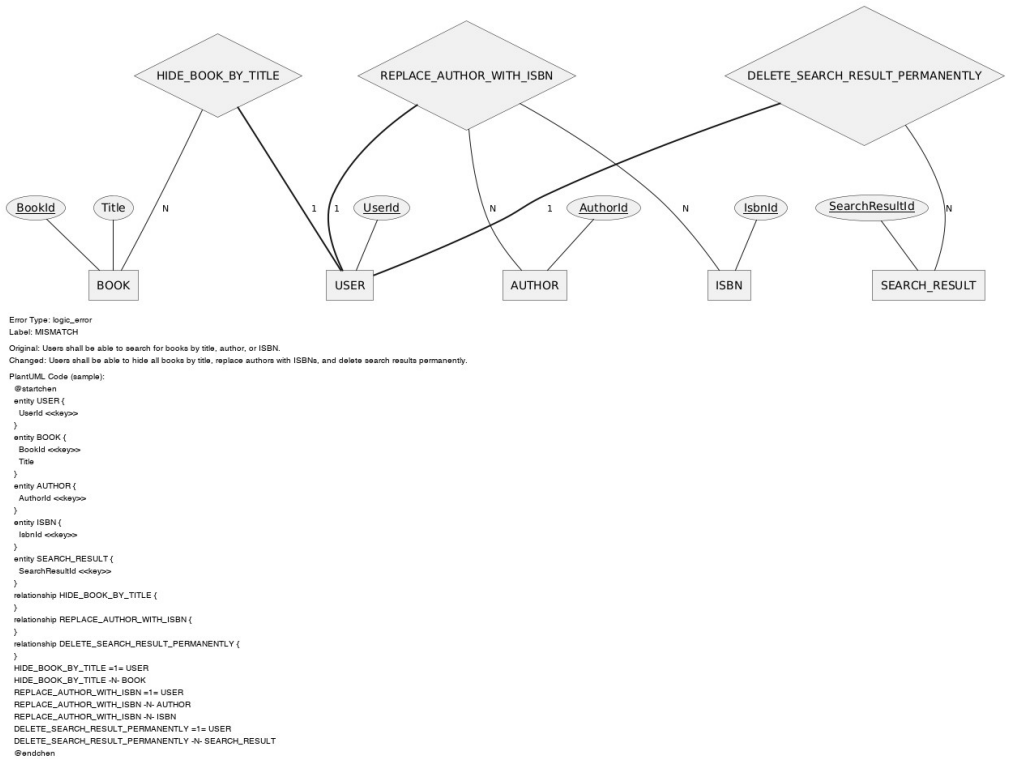


Figure 6. Example of Dataset C, Mismatch

3. METHODOLOGY

In this section, we are going to describe the four methods that are implemented to automatically check the consistency between a natural language requirement and an ER diagram written in PlantUML (which uses Chen's notation). All methods take the same input pair and output a binary label: MATCH or MISMATCH. The methods are designed with increasing levels of semantic and structural understanding and with each method, we tried appending our overall results by levelling up the performance to make more use of the logics to produce our best possible results for the input datasets. In some of the methods, we have deliberately expected not so top-notch results, just to keep it real and to make this mini project as realistic as possible, when compared to the real-world scenarios.

3.1 Method 1: Basic Rule-Based Matching:

The first method serves as a simple baseline. It relies on basic keyword extraction and character-level string similarity to determine whether important terms from the requirement appear in the ER diagram. The approach works as follows. First, the requirement text is cleaned and tokenized to extract key content words. Similarly, entity names and attribute names are extracted from the PlantUML ER diagram. So, with both these extractions, it will be done in a such a way, such that the mistakes will be at their minimal, so that the further steps can get benefitted from these results from the extraction that we are getting done with this step. For each requirement term, a similarity score is computed against diagram terms using a character-based similarity measure. If a sufficient proportion of requirement terms have high similarity with diagram terms, the pair is classified as MATCH; otherwise, it is classified as MISMATCH.

With this method, we have tried to produce two types of outputs, the first one being the visual output, which includes plotted graphs of the output, in this Fig. 8, shows us, this method very well works on Dataset A with the total of 75% of matching effciancy, and dataset B with 10%, and Dataset C giving us around 20% of matching accuracy, and the textual console output i.e., Fig 7, giving us the same results in a simple text format, and the second one being the csv files including the predictions, which includes each predictions for every data in the dataset . This method is fast, easy to implement, and fully interpretable. However, it is extremely sensitive to naming variations. Even meaningful synonym changes can cause failures, and the method has no understanding of entity roles, relationships, or logical structure.

```
...
 Method 1- Rule based results
Accuracy A: 75.00 %
Accuracy B: 10.00 %
Accuracy C: 20.00 %
----------------------------------
False Alarm A: 25.00 %
False Alarm B: 90.00 %
Logic Error Recall C: 20.00 %
----------------------------------
Saved: predictions_A_basic.csv
Saved: predictions_B_basic.csv
Saved: predictions_C_basic.csv
```

Figure 7. Console output of the Method 1 [Basic Rule based matching]

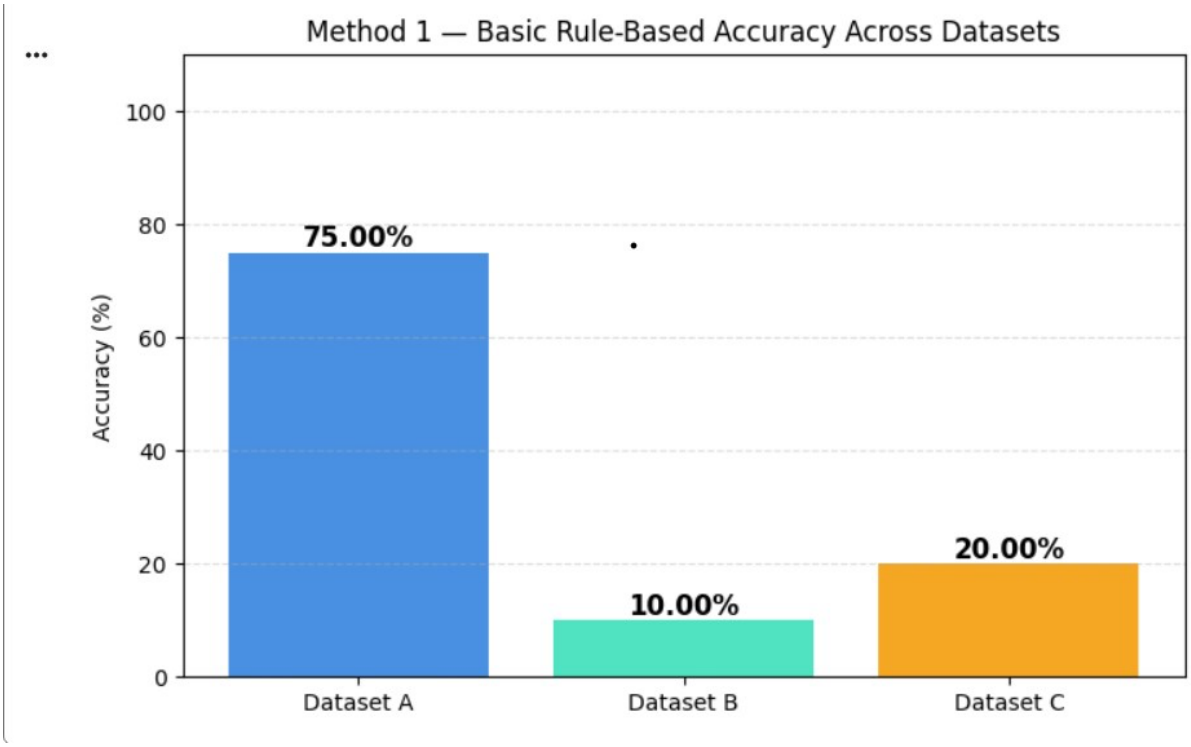


Figure 8. Graph output of the Method 1 [Basic Rule based matching]

3.2 Method 2: Advanced Rule-Based Matching:

The second method extends the basic rule-based approach with additional heuristics to improve robustness while remaining fully rule-based and interpretable. Compared to Method 1, this method introduces three main enhancements. First, it uses a domain-specific synonym mapping to handle common variations such as customer/user, account/statement/summary, and transaction/purchase. Second, it performs cleaner extraction of entity and relationship names from Chen-style PlantUML diagrams, including splitting compound identifiers.

Third, it applies lightweight structural sanity checks: if a requirement explicitly mentions key actors or objects (e.g., customers or accounts) but the ER diagram does not contain any corresponding entities, the method Favors a MISMATCH decision.

With this method, we have tried to produce two types of outputs, the first one being the visual output, which includes plotted graphs of the output, in this Fig. 9, shows us, this method very well works on Dataset A with the total of 90% of matching effciancy, and dataset B with 20%, with the slightest improvement than method 1, and Dataset C declining to 5% of matching accuracy, which demonstrates strict conditional prediction, and the textual console output i.e., Fig 10, giving us the same results in a simple text format and the textual console output, and the second one being the csv files including the predictions. Despite these improvements, Method 2 is still fundamentally string based. Synonym coverage is limited and manually defined, and the method cannot reason about cardinality, relationship direction, or deeper logical constraints. Nevertheless, it reduces false alarms compared to Method 1 and provides a stronger rule-based baseline.

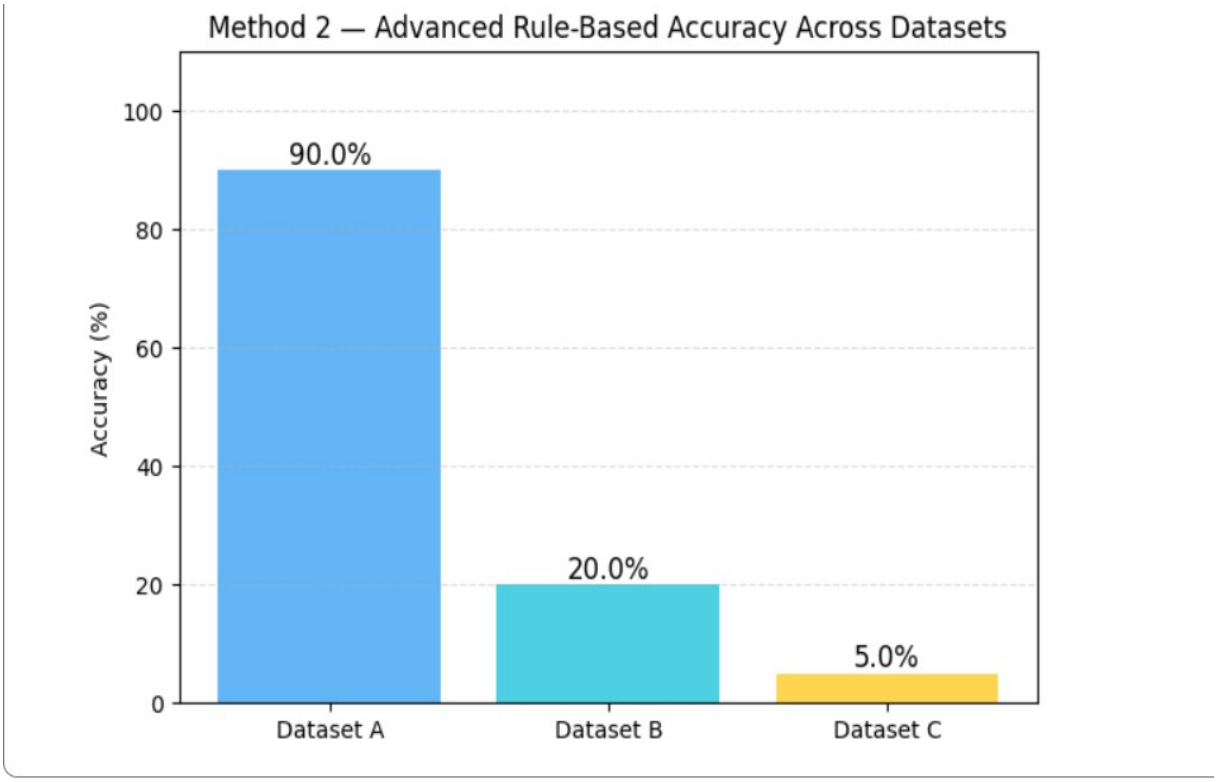


Figure 9. Graph output of the Method 2 [Advanced Rule-Based Matching]

```
METHOD 2 — ADVANCED RULE-BASED RESULTS
Accuracy A: 90.00%
Accuracy B: 20.00%
Accuracy C: 5.00%
----------------------------------------
False Alarm A: 10.00%
False Alarm B: 80.00%
Logic Error Recall C: 5.00%
----------------------------------------
Saved: predictions_A_advanced.csv
Saved: predictions_B_advanced.csv
Saved: predictions_C_advanced.csv
```

Figure 10. Console output of the Method 2 [Advanced Rule-Based Matching]

3.3 Method 3: Embedding-Based Semantic Matching:

The third method introduces semantic similarity using sentence embeddings. Instead of relying solely on surfacelevel string matching, this approach compares the meaning of the requirement and the ER diagram.

In this method, the natural language requirement is encoded into a dense vector using a pre-trained SentenceBERT model. The ER diagram is first converted into a simplified English description by listing entities and relationships in sentence form. This textual representation is then encoded using the same embedding model. The semantic similarity between the requirement and diagram embeddings is computed using cosine similarity. If the similarity score exceeds a predefined threshold, the pair is labelled as MATCH; otherwise, it is labelled as MISMATCH. To avoid unrealistic behaviour, a lightweight logic-cue detector is added, we could have made it at a higher place, where the predictor could have gone and be at their higher efficiency, where they can predict the latter side, where the slightest difference would have been counted, but we have decided it to look like a realistic scenario, and to obtain the results at its best. If the requirement contains strong negation or contradiction cues (for e.g., let's look at some of these negations-based words, “must not”, “cannot”, “prevent”), the method directly predicts MISMATCH.

In this method Fig. 12, shows us, this method increasing the efficiency of the prediction on Dataset A with the total of 90% of matching effciancy, and dataset B with 30%, which is high until is this point, than latter two methods, and Dataset C giving us around 15% of matching accuracy, and the textual console output i.e., Fig 11, giving us the same results in a simple text format, with the constraints like accuracy of all three datasets and listing the threshold value which we used to decide, which suits the best real world scenario and which gives us best estimates on the predictions.

This method is more robust to synonym and paraphrase variations than rule-based approaches. However, embeddings lack explicit structural and logical understanding, and the conversion from ER diagrams to English is approximate and lossy.

```
Saved: predictions_A_embed_final.csv
Saved: predictions_B_embed_final.csv
Saved: predictions_C_embed_final.csv

method3_embedded results
Threshold: 0.58
Accuracy A: 90.0%
Accuracy B: 30.0%
Accuracy C: 15.0%
----------------------------------------
False Alarm A: 10.0%
False Alarm B: 70.0%
Logic Error Recall C: 15.0%
----------------------------------------
```

Figure 11. Console output of the Method 3 [Embedding-Based Semantic Matching]

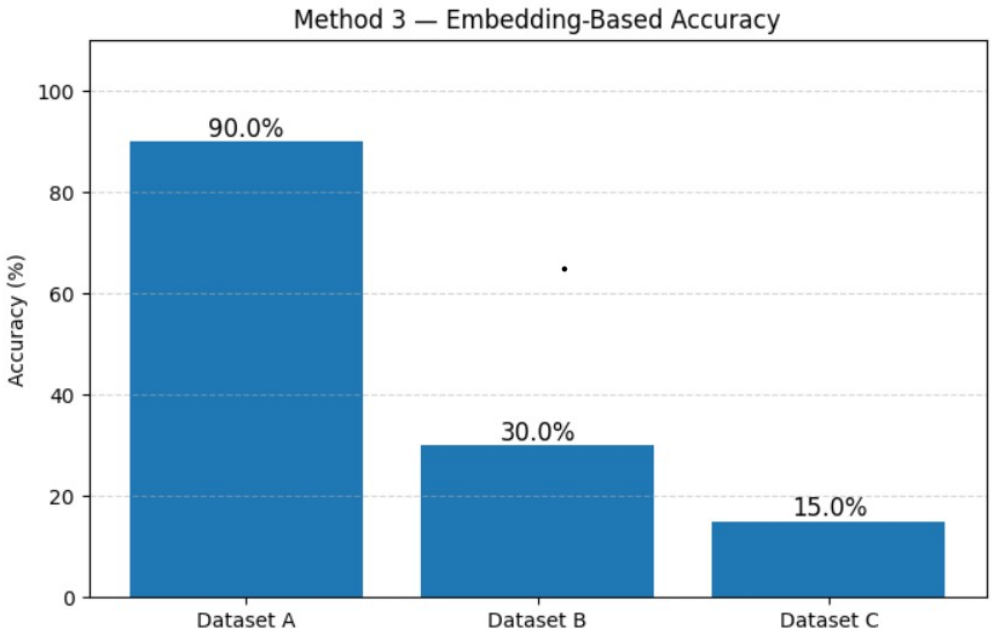


Figure 12. Graph output of the Method 3 [Embedding-Based Semantic Matching]

3.4 Method 4: LLM-Based Semantic and Structural Reasoning:

The fourth method uses a large language model (LLM) to perform joint semantic and structural reasoning. Given the requirement text and the full PlantUML ER diagram, the model is prompted to analyse whether the diagram satisfies the intent of the requirement. In this method, we have made use of Open AI model to get us the result, in which we are trying to impart that, how a large learning model can impact the overall output quality and the level of perfection that it can achieve is on terms of its own, and this is the important point that we are trying to make it here. So, we have used an API agent to bridge our connection between the environment, in which we are code, and the input elements exist and to the LLM model, which exists in the cloud space of its own. So, in human terms, we are summoning a ChatGPT model to explore the dimensions, by taking our input into considerations, and then in our code, we have given the model a prompt to guide it in a right direction, so that it fetches us the right results.

Unlike previous methods, the LLM considers entities, relationships, roles, and implied logic simultaneously. It produces a final decision (MATCH or MISMATCH) along with a short explanation describing the reasoning behind the decision, which we can get it through its environment, but for now, we have asked the model to generate us a clear CSV file with predictions in it, so that the output files that we generate from this method stays in similarity with the output files, from the other three methods, so that it would be easy for us to compare the outputs overall from all four methods.

In this method Fig. 14, shows us, this method performing nest on the efficiency of the prediction over Dataset A with the total of 95% of matching effciancy, which is the best accuracy so far of all methods, delivering us with the almost realistic predictions which encourages us to do more work on these topic, and dataset B with 70%, which is high until is this point, and this level of predictions shows us the power of large learning models, where irrespective of the synonyms differences, it almost predicts most of the things right, which is so far the best work on dataset B, than latter two methods, and the Dataset C giving us around 55% of matching accuracy, this displays us that the intelligence and the level of analysis a LLM model can perform on the given inputs, where its analysed structural and logical challenges with utmost grace, and it opens us an opportunity that this LLM support combined with the very less human effort can produce us fault proof system, which could be path breaking. The textual console output i.e., Fig 13, giving us the same results in a simple text format, with the constraints like accuracy of all three datasets and listing the other factors, which suits the best real-world scenario, and which gives us the best estimates on the predictions.

This method achieves the strongest overall performance, especially for detecting logical inconsistencies. That is with reference to the dataset C, which was one of the hardest datasets to solve amongst the other two, because it had some logical and structural things which demanded intelligence to put on, to detect or predict the match or mismatch between the comparison. However, it has higher computational cost and latency and depends on access to an external API. Therefore, it is used as an upper-bound reference rather than a lightweight solution.

```
Evaluating Accuracy: 100%|██████████| 20/20 [00:18<00:00,  1.06it/s]
Evaluating Accuracy: 100%|██████████| 20/20 [00:11<00:00,  1.70it/s]
Evaluating Accuracy: 100%|██████████| 20/20 [00:09<00:00,  2.09it/s]
False Alarm Calc: 100%|██████████| 20/20 [00:11<00:00,  1.69it/s]
False Alarm Calc: 100%|██████████| 20/20 [00:08<00:00,  2.27it/s]
Logic Recall Calc: 100%|██████████| 20/20 [00:09<00:00,  2.04it/s]

===== METHOD 4 RESULTS (LLM-BASED) =====
Accuracy A: 95.00 %
Accuracy B: 70.00 %
Accuracy C: 55.00 %
----------------------------------------
False Alarm A: 5.00 %
False Alarm B: 30.00 %
Logic Error Recall C: 45.00 %
Predicting predictions_A_llm.csv: 100%|██████████| 20/20 [00:11<00:00,  1.76it/s]
Saved: predictions_A_llm.csv
Predicting predictions_B_llm.csv: 100%|██████████| 20/20 [00:10<00:00,  1.90it/s]
Saved: predictions_B_llm.csv
Predicting predictions_C_llm.csv: 100%|██████████| 20/20 [00:12<00:00,  1.65it/s]Saved: predictions_C_llm.csv
```

Figure 13. Console output of the Method 4 [LLM-Based Semantic and Structural Reasoning]

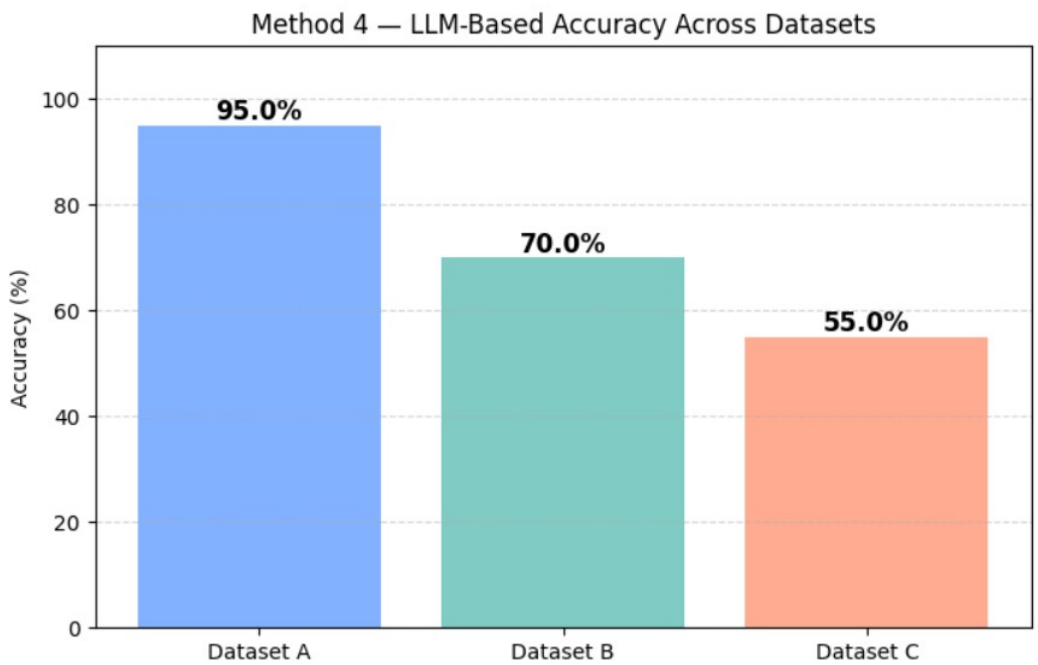


Figure 14. Graph output of the Method 4 [LLM-Based Semantic and Structural Reasoning]

4. EVALUATION

All four methods are evaluated on the same three datasets: Dataset A (PERFECT), Dataset B (SYNONYM), and Dataset C (LOGIC_ERROR). Each dataset contains 20 requirement–diagram pairs. We report accuracy for each dataset, along with false alarm rates for Datasets A and B, and logic error recall for Dataset C. So, here we considered every possible cause case, before proceeding with the evaluation, and we considered the what the datasets type was, and what each dataset type is bringing to the evaluation, and later we proceeded with the prediction factors, analysis and developmental steps that needs to be taken to evaluate.

So, here dataset A represents ideal conditions where requirements and diagrams match exactly, which will be easy for a developed method to compare and find flaws between the stage in comparison. While, Dataset B tests robustness to synonym and naming variation, where it challenges the factors like analysis of each keyword, what it means, how the extracted text should be compared, and with what it should be compared, to really know whether the resulted text is a synonym or not of a keyword with which we are comparing. Dataset C evaluates the ability to detect logical inconsistencies, so this method demanded some intelligence to detect, analyse, compare and in predicting whether the given keywords from both the text and diagram are in sync or not, so this method was a real challenge. The results show clear trends. Method 1 performs reasonably on Dataset A but fails significantly on Dataset B due to its sensitivity to wording changes. Method 2 reduces false alarms and slightly improves performance on synonym-modified requirements but still struggles with logical errors. Method 3 demonstrates stronger semantic matching and improves robustness to paraphrasing but remains limited in detecting deep logical inconsistencies. Method 4 achieves the best overall performance and is the most reliable for identifying structural and logical mismatches.

These results confirm that increasing semantic and reasoning capability improves robustness but also increases computational cost.

5. CONCLUSION

In this mini project, we studied the problem of automatically verifying consistency between natural language requirements and ER diagrams using multiple approaches, where it challenged us in elevating its potentials, so we designed a roadmap pf possible steps that might help us in solving this precise input datasets and getting us near to the expected output, that we were anticipating. Using a small but carefully constructed dataset, we evaluated four different methods with increasing complexity, the four different methos would be Basic Rule-Based Matching, Advanced Rule-Based Matching, Embedding-Based Semantic Matching, LLM-Based Semantic and Structural Reasoning.

Starting from simple rule-based matching and progressively incorporating synonym handling, semantic embeddings, and large language models, we demonstrated the strengths and limitations of each technique. Our results show that rule-based methods are fast and interpretable but highly sensitive to naming variations. The second method extends the basic rule-based approach with additional heuristics to improve robustness while remaining fully rule-based and interpretable. Compared to Method 1, this method introduces three main enhancements, something like Extraction, comparison, identification. while embedding-based methods improve semantic robustness but struggle with logical correctness. So, the Embedding-based methods mainly improve semantic robustness but lack explicit logical understanding. The final method, which is the LLM-based reasoning provides the strongest performance and explanations, highlighting its potential for future automated design validation tools.

Overall, this work demonstrates both the challenges of requirement–diagram consistency checking and the promise of modern NLP and LLM techniques in supporting software design validation

REFERENCES

[1] Wiegers, Karl, and Joy Beatty. Software requirements. Pearson Education, 2013.

[2] Chechik, Marsha, and John Gannon. "Automatic analysis of consistency between requirements and designs." IEEE transactions on Software Engineering 27.7 (2002): 651-672.

[3] Hou, Xinyi, et al. "Large language models for software engineering: A systematic literature review." ACM Transactions on Software Engineering and Methodology 33.8 (2024): 1-79.

[4] Chen, Peter Pin-Shan. "The entity-relationship model—toward a unified view of data." ACM transactions on database systems (TODS) 1.1 (1976): 9-36.

[5] Marques, Nuno, Rodrigo Rocha Silva, and Jorge Bernardino. "Using chatgpt in software requirements engineering: A comprehensive review." Future Internet 16.6 (2024): 180

[6] OpenAI. “API Reference.” OpenAI Platform, n.d., https://platform.openai.com/docs/apireference/introduction. Accessed 11 Dec. 2025.

[7] codinganirbanb. “Requirement Specifications to Generate ER Diagrams.” Kaggle, n.d., https://www.kaggle.com/datasets/codinganirbanb/requirement-specifications-to-generate-er-diagrams. Accessed 11 Dec. 2025.

[8] “PlantUML.” PlantUML, n.d., https://plantuml.com/. Accessed 11 Dec. 2025.

[9] “PlantUML PNG Endpoint.” Kroki, n.d., https://kroki.io/plantuml/png. Accessed 11 Dec. 2025.